

Select and Configure Compute — Cheat Sheet

Microsoft Learn module: Select and configure compute in Azure Databricks (Unit 4)

1. Choosing a Compute Type

Serverless compute

- Fully managed by Databricks — runs in **Databricks' subscription**, not yours; no VMs/networking appear in your Azure subscription.
- Startup: **2–6 seconds**. Scales up/down automatically; requires **Unity Catalog**.
- Available for: notebooks, jobs, Lakeflow Spark Declarative Pipelines, SQL warehouses.
- Versionless runtime** — Databricks applies upgrades automatically, no migration effort.
- Limits: no Scala/R notebooks, no JAR libraries, no RDD APIs, restricted custom Spark configs. JAR tasks in Lakeflow Jobs on serverless = Public Preview.

Serverless performance modes (jobs & pipelines)

Mode	Startup	Cost	Use for
Performance-optimized (default)	Seconds — warm pool	Higher DBU usage	Interactive/latency-sensitive jobs
Standard	~4–6 minutes	Up to 70% less DBU	Scheduled batch jobs/pipelines where latency is OK

- Toggle via **Performance optimized** on the job details page. Standard mode isn't available for continuous pipelines, one-time runs/submit calls, or notebooks (notebooks = performance-optimized only).

Classic compute

- Runs in **your** Azure subscription — full control, more management. Startup: **3–7 minutes**. You manually choose/upgrade the Databricks Runtime version.
- Standard access mode**: multiple users share one cluster; **Lakeguard** isolates user code. Good for collaborative engineering/shared analytics.
- Dedicated access mode**: cluster assigned to one user/group; full machine-level privileges — required for RDD APIs, GPU workloads, R language, custom containers.
- Multi-node** (driver + 1+ workers): horizontal scaling, production/large-data workloads.
- Single-node** (driver only, no workers): no distribution — fits lightweight exploration, small datasets, ML experimentation (e.g. scikit-learn) where shuffle overhead isn't worth it.

SQL warehouses

Type	Photon	Predictive IO	Intelligent Workload Mgmt	Startup	Best for
Serverless	✓	✓	✓	2–6 sec	Most SQL: BI, ETL, ad hoc
Pro	✓	✓	—	~4 min	Custom networking / federation
Classic	✓	—	—	~4 min	Entry-level exploration only

Instance pools

- Keep idle VM instances warm; creating a cluster from a pool cuts startup to **under a minute** in many cases.
- You pay VM cost while idle, but not Databricks compute units (DBUs). Cost-effective for frequently recreated classic clusters.

- With serverless available, pools matter less overall — but still useful when you need classic-only features and fast startup.
- Tip: use **spot instances for workers** in a pool to cut cost, but keep **on-demand for drivers** for reliability.

Job compute

- Optimized for automated workflows; **terminates automatically** after tasks complete — no idle cost.
- **Serverless job compute**: faster startup, fully managed, usually cheaper for automated workloads.
- **Classic job compute**: more configuration options — autoscaling, spot instances (up to ~90% cheaper, Azure can reclaim with only 30 seconds notice; Spark retries failed tasks on healthy nodes automatically).
- The built-in **Job Compute policy** gives a production-ready template enforcing the latest LTS runtime and reliability defaults.

Decision guide

- **Default to serverless** for almost everything — least ops overhead, best cost/scaling, fastest iteration.
- Switch to **classic with dedicated access** only when you specifically need RDD APIs, R, GPU, or custom containers.
- Need custom VNets / on-prem connectivity / specific instance types? → classic compute or Pro SQL warehouses (serverless has no presence in your subscription's network).
- Frequent, recreate-often classic workloads? Consider **instance pools** to cut startup time.

2. Configuring Compute Performance

- **Total executor cores** = max parallelism (e.g. 8 workers × 4 cores = 32 cores total).
- **Total executor memory** = how much fits in-memory before spilling to disk (spilling significantly slows joins/aggregations).
- **Local storage** = temp space for shuffle operations and caching — faster local disk speeds up shuffles.

Node/instance families

Family	Optimized for	Example VMs
Memory-optimized	Large joins/aggregations, data that should stay in memory	E-series
Compute-optimized	Complex calculations, straightforward ETL, lower memory need	F-series
Storage-optimized	Repeated reads of same data, fast local disk / caching	L-series (NVMe)
GPU-accelerated	ML/deep learning training (10–100x faster than CPU-only); requires Databricks Runtime ML series (NVIDIA)	NC-series, ND-series (NVIDIA)

- Fewer, larger workers → less shuffle network traffic → better for analytical/shuffle-heavy workloads.
- More, smaller workers → better parallelism for highly distributed, simple batch workloads.

Flexible node types

- Solves CLOUD_PROVIDER_RESOURCE_STOCKOUT errors by auto-falling back to compatible instance types (same vCPUs, memory within 100–110%, disk config, CPU architecture, OS image support).
- Enable via workspace Compute admin settings (**Enable auto flexible node types**) — applies to all new classic compute.
- Especially valuable with **spot instances**: tries multiple compatible types before falling back to on-demand, raising the odds of getting (cheaper) spot capacity.
- Disable per-resource by setting `alternate_node_type_ids` to an empty list via the Clusters API, or supply your own custom fallback list.

Autoscaling

- Set min/max worker counts; Databricks adds/removes workers within that range as demand changes.
- **Optimized autoscaling** (default): scales up in two quick steps to max; can scale *down* even when not fully idle by watching shuffle file state.
- Utilization checks: every **40 seconds** for job compute, every **150 seconds** for all-purpose compute.
- Best for variable workloads (e.g. exploration starting small, then processing larger data). For steady, predictable workloads, a fixed worker count is often more stable.
- Pairs well with instance pools: set min workers $\leq$ the pool's min idle instances for instant scale-up.

Termination & spot instances

- **Automatic termination**: set an inactivity period (e.g. ~45 min for interactive work); cluster config stays available for restart. Job compute auto-terminates after the job finishes.
- **Spot instances**: fine for *worker* nodes (Spark tolerates failures) — never for the *driver* (losing it kills the whole cluster).
- Enable **decommissioning** with spot workers: on a preemption notice, shuffle/cached data migrates to healthy workers first, avoiding recompute.

Instance pool tuning

- Set **minimum idle instances** to match typical concurrent cluster needs (e.g. 3 idle if you usually run 3 notebooks at once).
- Set **maximum capacity** to cap cost / ensure fair sharing across teams (e.g. split a 100-instance quota into two 50-instance pools).
- **Idle instance auto-termination**: instances above the minimum are released after the configured idle period.
- **Preload a Databricks Runtime** on pool instances for near-instant cluster launches.
- Best for frequent create/terminate cycles (dev teams); less benefit for long-running dedicated production clusters.

Use the Spark UI to diagnose performance: check the Jobs Timeline for long-running stages, then the stage details page for **Shuffle Spill (Memory)** and **Shuffle Spill (Disk)** statistics.

3. Configuring Compute Features

Photon

- Native-code query engine replacing standard Spark execution for SQL/DataFrame ops — big wins on joins, aggregations, large scans.
- Enabled by default on **Databricks Runtime 9.1 LTS+**. Configurable under Performance during compute creation.
- **Not supported on GPU-enabled clusters** — must disable Photon for ML/GPU workloads.
- Minimal benefit for simple, sub-2-second batch ETL jobs — may not be worth the extra cost there.

Runtime & Spark version

- Interactive/all-purpose compute: use the **latest runtime** for newest features/compatibility.
- Job compute (production): prefer **LTS (Long Term Support)** versions for stability — test before upgrading.
- Spark version is tied to the runtime you choose — picking a runtime picks the Spark version.

ML environments

- Use **Databricks Runtime ML** — preinstalled ML libraries, GPU drivers, CUDA for deep learning.

- Single-node with a large instance often suffices for initial experimentation (minimizes shuffle overhead).
- GPU instances accelerate image recognition/NLP/neural net training dramatically — but again, **Photon must be disabled** when using GPU instances.

4. Installing Libraries

- **Compute-scoped libraries** install on a cluster and are available to every notebook/job on it; reinstalled automatically on every cluster start. Requires **CAN MANAGE** permission on the cluster.
- Support Python wheels, JAR files, and R packages — from repositories (PyPI, Maven, CRAN) or from files (workspace files, Unity Catalog volumes, cloud storage).
- Limitation: a cluster-scoped library affects every notebook on that cluster — conflicting version needs across teams require separate clusters or notebook-scoped installs.

From package repositories

- **PyPI:** pin an exact version for production, e.g. `pymssql==2.3.9` — no version pin means "latest," which can silently change and break code.
- **Maven:** coordinates as `groupId:artifactId:version` (e.g. `com.microsoft.sqlserver:mssql-jdbc:13.2.1.jre11`); supports excluding transitive dependencies.
- **CRAN (R):** always installs the latest version — to pin an R package version, store the files in workspace files/volumes instead.
- On **standard access mode** clusters, Maven coordinates and JAR paths require **allowlist approval** first.

From files

- **Workspace files:** 500 MB size limit; reference via a path like `/Workspace/Users/you@example.com/libraries/mypackage-1.0.0-py3-none-any.whl`.
- **Unity Catalog volumes:** stronger governance (access control + audit logs); path like `/Volumes/main/engineering/libraries/mypackage-1.0.0-py3-none-any.whl` — installer needs **READ VOLUME**.
- **requirements.txt** works from workspace files or volumes on **Databricks Runtime 15.0+**, installing every listed package at once.
- Standard access mode requires allowlisting file paths too, same as Maven/JAR.

Init scripts

- Shell commands that run at cluster startup, *before* Spark starts — for system-level config (apt-get packages, env vars, monitoring agents), not for library installs (cluster-scoped libraries are the better tool for that).
- Store in **Unity Catalog volumes** for Runtime 13.3 LTS+; run in the order specified; a non-zero exit code fails cluster startup (protects against broken config).
- Standard access mode requires the init script path to be allowlisted separately from JAR/library paths.

Allowlisting (standard access mode)

- A **metastore admin** must allowlist Maven coordinates (`groupId:artifactId:version`, or partial for broader scope) and JAR/init-script paths before use.
- Path allowlisting uses **prefix matching** — e.g. `/Volumes/prod-libraries/` permits everything under it; include the trailing slash to avoid unintended matches.
- Allowlisting only grants permission to use the path — the installer still needs data-access permissions (e.g. **READ VOLUME**).
- Configure via Catalog Explorer → metastore settings → Allowed JARs/Init Scripts.

5. Configuring Compute Access

Level	Grants
NO PERMISSIONS	Can't see, attach to, or interact with the compute at all (default/private)
CAN ATTACH TO	Attach notebooks, view Spark UI and metrics — no start/stop/restart
CAN RESTART	Everything above + start/stop/restart the compute lifecycle
CAN MANAGE	Everything above + edit config, attach libraries, resize, modify others' permissions

- Workspace admins get **CAN MANAGE** on everything automatically; the creator of a compute resource becomes its owner with CAN MANAGE.
- Only **CAN MANAGE** users can view driver logs on job/dedicated/standard clusters by default — logs can expose secrets via stdout/stderr. (Adjustable via `spark.databricks.ac1.needAdminPermissionToViewLogs`, but loosening this is a security tradeoff.)
- ■■ **No isolation shared** access mode is legacy — users with only CAN ATTACH TO can view service account keys in log4j files on this mode. Avoid it; use Standard or Dedicated instead.
- Set permissions from the workspace UI (Compute → resource → Permissions), not the Azure portal — this is independent of Azure subscription-level RBAC. Changes apply immediately, no restart needed.
- Prefer granting permissions to **groups** over individuals — scales better as teams change.

Workspace entitlements

- **Unrestricted cluster creation** entitlement lets non-admins create clusters of any size, without granting full workspace admin rights (principle of least privilege).
- Grant via Settings → Identity and access → Users/Groups → Entitlements → **Allow cluster creation**.

Dedicated group access mode

- UI default access mode is **Auto**: picks Standard unless you choose Runtime ML or a runtime below 14.3, in which case it auto-selects **Dedicated**.
- To assign a cluster to a specific group, explicitly choose **Dedicated** (formerly "Single-user") in the Advanced section, and pick the group.
- Requires **Unity Catalog enabled** and **Databricks Runtime 15.4+**. The group needs CAN MANAGE on a workspace folder (convention: `/Workspace/Groups/<groupName>`) for notebooks/experiments.
- On a group cluster, individual permissions scope down to the group's permissions — enables secure sharing for ML runtimes, RDD APIs, and R without privilege escalation.
- Data objects created on a group cluster are **owned by the group**, not the individual user (e.g. `CREATE SCHEMA` run on a group cluster → the group owns the schema).

Best practices

- Apply least privilege: CAN ATTACH TO for analysts, CAN RESTART for engineers managing availability, CAN MANAGE reserved for owners/admins.
- Use groups mirroring your org structure rather than individual grants.
- Restrict production compute tightly — CAN MANAGE only for the owning team.
- Regularly audit access via the `system.access.audit` table; remove access for people who've left a project.
- Never use No isolation shared mode — migrate any existing clusters on it to Standard or Dedicated.